第一章

1.计算机组成和计算机体系结构

1.1 计算机体系结构

1) 概念 : 指那些能够被程序员所见到的计算机系统的属性，即概念性的结构与功能特性。

2) 计算机系统的属性通常是指用机器语言编程的程序员所看到的传统机器的属性，包括指令集，数据类型，存储器寻址技术，I/O 机理等。(大都为抽象的属性)

1.2 计算机组成

1) 概念：实现计算机体系结构所体现的属性。

2.计算机层次结构

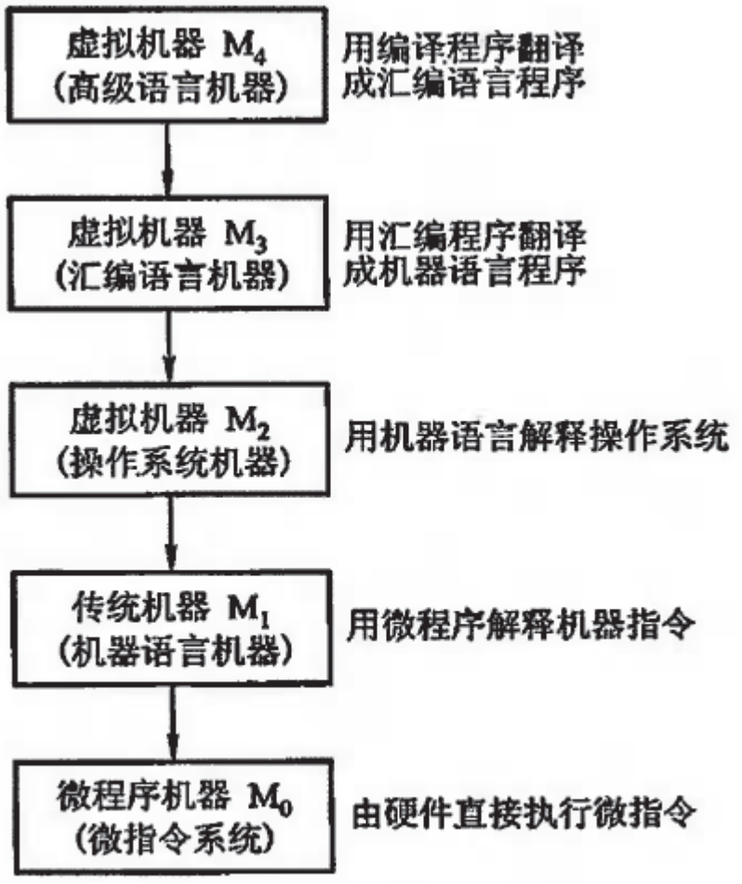


图 1.6　多级层次结构的计算机系统

3.**冯诺依曼计算的特点**

1）计算机由五大部件组成

2）指令和数据以同等地位存于存储器，可按地址寻访

3) 指令和数据用二进制表示

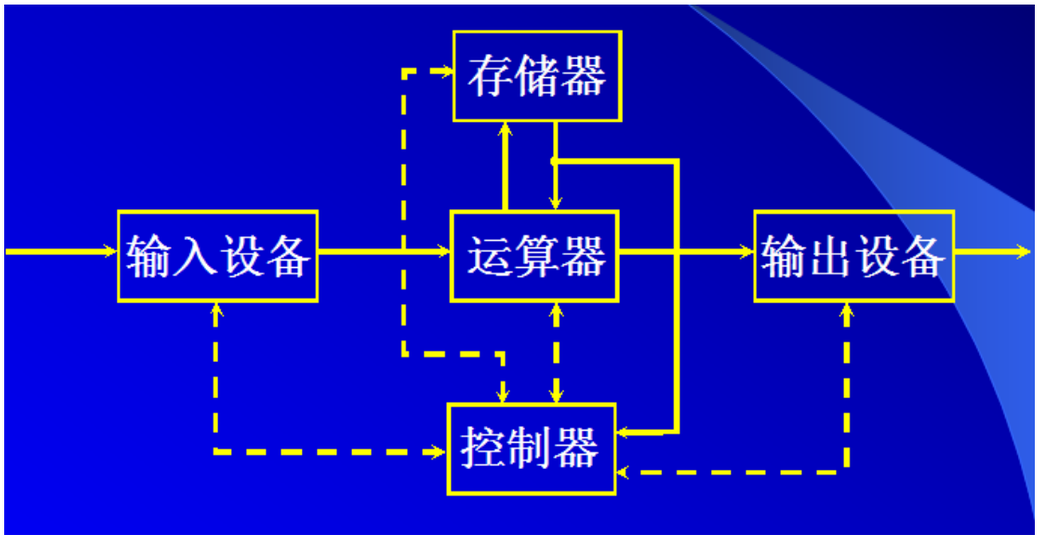
4) 指令由操作码和地址码组成

5) 存储程序

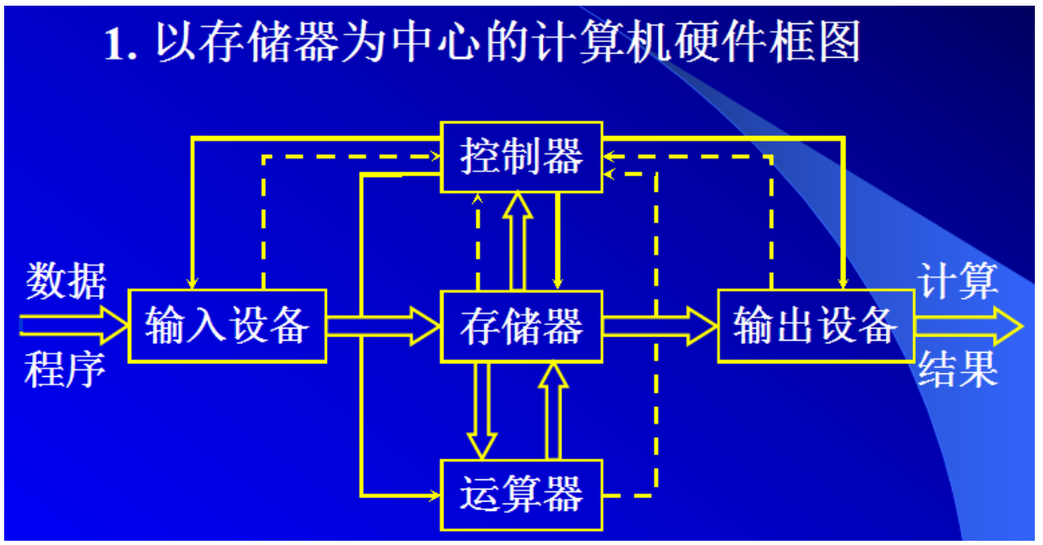
6) 机器以运算器为中心

4.计算机硬件框图

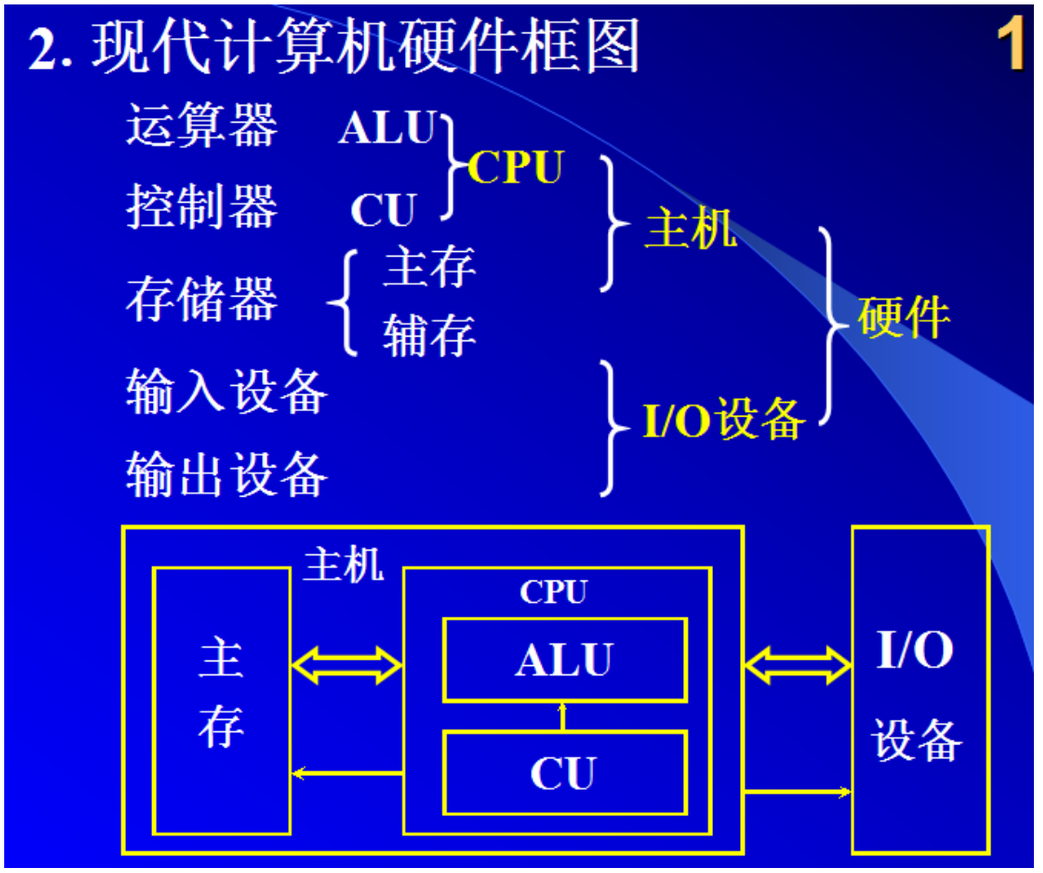
1）冯. 诺依曼计算机结构框图



2）现代以存储器为中心的计算机结构框图


1. 以存储器为中心的计算机硬件框图
控制器
数据
程序
输入设备
存储器
输出设备
计算
结果
运算器

3)


2. 现代计算机硬件框图
1
运算器
ALU
控制器
CU
CPU
主机
存储器
主存
辅存
硬件
输入设备
输出设备
I/O设备
主机
主存
CPU
ALU
CU
I/O
设备

5.计算机硬件的主要技术指标

1）**机器字长** ：CPU 一次能够处理数据的位数，与 CPU 中的 寄存器位数 有关。

2）**存储容量**：主存容量与辅存容量

主存容量是指主存中存放二进制代码的总位数。即

存储容量 = 存储单元个数 * 存储字长

MAR 位数代表存储单元个数，MDR 的位数代表存储字长。

例:MAR = 10，则存储单元个数=2^{10}，MDR=8，即有 8 位。

字节数：2^{13} = 1KB （1K = 2^{10}，1B = 2^3b）

辅存容量：通常用字节数来表示，如 80GB（1G=1024M=2^{10} * 2^{20}=2^{30}）

3）**运算速度**：单位时间内执行指令的平均条数

计量单位：MIPS（百万条指令每秒）

CPI（执行一条指令的时钟周期（机器主频的倒数））与 FLOPS（浮点运算次数每秒）也可来衡量运算次数。

第二章

1.计算机产生和发展过程

1）计算机的 3 代：电子管——晶体管——集成电路

2）硬件技术对计算机更新换代的影响

代	时间	硬件技术	速度 /（次/秒）
一	1946—1957	电子管	40 000
二	1958—1964	晶体管	200 000
三	1965—1971	中小规模集成电路	1 000 000
四	1972—1977	大规模集成电路	10 000 000
五	1978—现在	超大规模集成电路	100 000 000

2.什么是摩尔定律

微芯片上集成的晶体管数目每三年翻两番

第三章

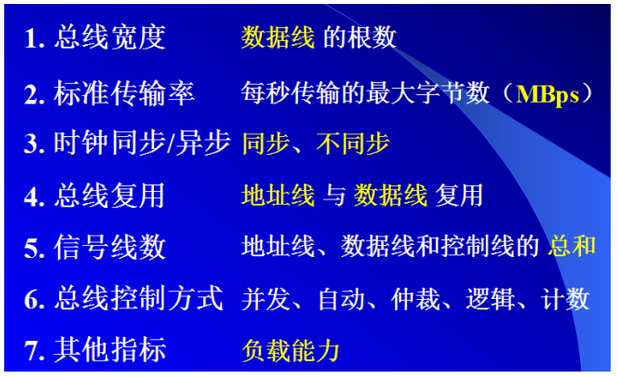
1 总线性能指标

①总线宽度：指数据总线的宽度（bit 为单位）

②总线带宽：总线的数据传输速率（MBps 为单位）

总线带宽 = 总线宽度 * 总线频率

例：总线工作频率为 33MHz,总线宽度为 32 位，则带宽=33*（32/8）=132MBps.



2.ISA，EISA、PCI，RS-232C 与 USB 总线特点

ISA：又称 AT 总线，使用独立于 CPU 的总线时钟，不支持多台主设备系统，而且 ISA 上的所有数据传送需通过 CPU 或 DMA 接口来管理，耗 CPU 时间.

最大传输速率（带宽）：16MBps 总线时钟频率：8MHz 数据线： 16 地址线：24

EISA：与 ISA 完全兼容，从 CPU 中分离总线控制权，支持多个总线主控器与突发方式的传输。 最大传输速率（带宽）: 33MBps 总线时钟频率：8MHz 数据线： 32 地址线： 32

PCI：1.高性能 2.良好的兼容性 3.支持即插即用 4.支持多主设备能力

5.具有与处理器和存储器子系统完全并行操作的能力

6.提供数据与地、地址奇偶校验功能，保证数据的完整与准确。

7.支持两种电压标准：5V 与 3.

8.可扩充性好

9.软件兼容性好

10.采用多路复用技术，减少了总线引脚个数

RS-232C：是用于串行二进制 DTE（数据终端设备）与 DCE（数据通信设备）之间的标准接口。使用 RS-232C 时 必须实现两种电平的转换。即低电平为逻辑“1”(-15V~~-3V）

高电平为逻辑 “0”（ +3V~~+15V ）

USB：1.具有真正的即插即用的特征 2.具有很强的连接能力

3.数据传输率（USB1.0）有两种：普通无屏蔽双绞线（1.5MBps）和屏蔽双绞

线（12MBps） 而 USB2.0 数据传输率最高可达 480Mbps

4.标准统一　　5.连接电缆轻巧，电源体积缩小　　6.生命力强

总线标准	数据线	总线时钟	带宽
ISA	16	8 MHz（独立）	33 MBps
EISA	32	8 MHz（独立）	33 MBps
PCI	32 64	33 MHz（独立） 64 MHz（独立）	132 MBps 528 MBps
RS-232	串行通信总线标准	数据终端设备（计算机）和数据通信设备（调制解调器）之间的标准接口	
USB	串行接口总线标准	普通无屏蔽双绞线 带屏蔽双绞线 最高	1.5 Mbps (USB1.0) 12 Mbps (USB1.0) 480 Mbps (USB2.0)

3.三种集中式总线判优控制的特点

（1）链式查询　只需很少几根线就能按一定优先次序实现总线控制，并且很容易扩充设备，但对电路故障敏感，且优先级别低的设备很难获得请求。

（2）计数器定时查询　计数可以从‘0’开始，此时一旦设备的优先次序被固定，设备的优先级就按 0,1,2·n 的顺序降序排列，而且固定不表；计数也可以从上一次计数的终止点开始，既是一种循环方法，此时设备使用总线的优先级相等；计数器的初始值还可由程序设置，故优先次序可以改变。

（3）独立请求查询 响应速度快，优先次序控制灵活（可通过程序改变），但控制线数量多，总线控制更复杂

4.总线周期的四个阶段（完成一次总线操作的时间）

（1）申请分配阶段：主模块申请，总线仲裁决定

（2）寻址阶段：主模块 向 从模块给出地址和命令

（3）传数阶段：主模块和从模块交换数据

（4）结束阶段：主模块撤销有关信息

.P61 例 3.1

例 3.1 假设总线的时钟频率为 100 MHz，总线的传输周期为 4 个时钟周期，总线的宽度为 32 位，试求总线的数据传输率。若想提高一倍数据传输率，可采取什么措施？

解：根据总线时钟频率为 100 MHz，得

1 个时钟周期为 1/100 MHz = 0.01 μs

总线传输周期为 0.01 μs × 4 = 0.04 μs

由于总线的宽度为 32 位 = 4 B（字节）

故总线的数据传输率为 4 B/(0.04 μs) = 100 MBps

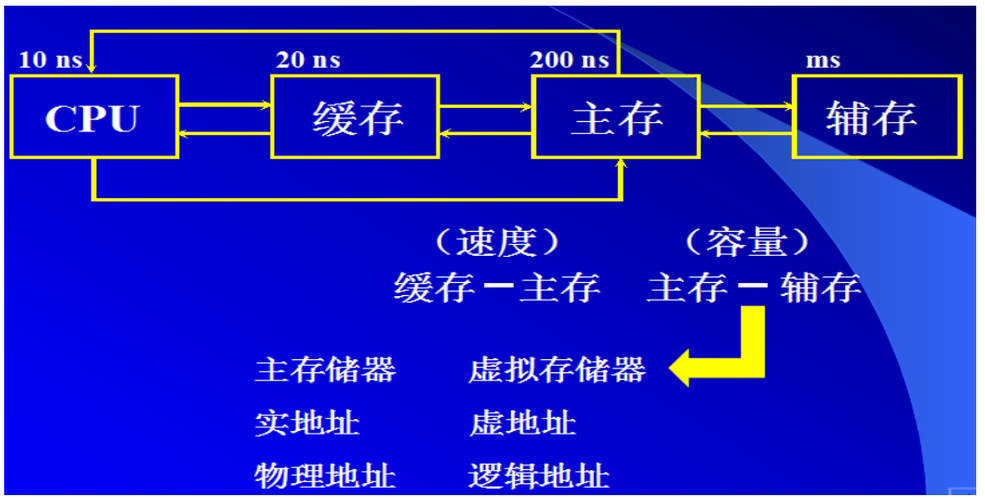
若想提高一倍数据传输率，可以在不改变总线时钟频率的前提下，将数据线的宽度改为 64 位，也可以仍保持数据宽度为 32 位，但使总线的时钟频率增加到 200 MHz。

第四章

1.存储器层次结构

存储器 3 个主要性能指标：速度，容量，位价。

存储器层次结构主要体现在**缓存----主存**和**主存----辅存**



缓存---主存层次主要解决 CPU 和主存速度不匹配的问题

主存---辅存层次主要解决存储系统的容量问题。

详解：CPU和主存，缓存都能直接交换信息；缓存能直接和CPU、主存交换信息；主存可以喝CPU、缓存、辅存交换信息。

Cache—主存层次在存储系统中主要对 CPU 访存起加速作用 ,即从整体运行的效果分析，CPU 访存速度加快，接近于 Cache 的速度，而寻址空间和位价却接近于主存。

主存—辅存层次在存储系统中主要起扩容作用，即从程序员的角度看，他所使用的存储器其容量和位价接近于辅存，而速度接近于主存。

综合上述两个存储层次的作用，从整个存储系统来看，就达到了速度快、容量大、位价低的优化效果。

2.**主存的技术指标**

（1）存储容量：主存中存放二进制代码的总位数。（上看第一章的硬件技术指标）

（2）存储速度：由存取时间和存取周期来表示。

（3）存储器带宽　**（存储容量和存储速度为主要技术指标）**

3.动态 RAM 的刷新方式

（1）集中刷新　（2）分散刷新　（3）异步刷新

4.ROM，SRAM，DRAM 和闪存的特点

ROM：（掩模 ROM：无法改变原始状态）

（PROM：实现一次编程，不再修改）

（EPROM：可擦除可编程）

SRAM：用触发器工作原理存储信息，即信息读出后，仍保持其原始状态，不得再生

DRAM:靠电容存储电荷的原理来存储信息

闪存:价格便宜、集成度高、可擦除重写。具有整片擦除的特点，擦除、重写的速度快。

5.存储器容量的扩展

定义：将若干存储芯片连在一起才能组成足够容量的存储器。

构成：（1）位扩展 （2）字扩展 （3）字、位扩展

在实际应用CPU与存储芯片时，还会遇到两者时序的配合、速度、负载匹配等问题，下面用一个实例来剖析CPU与存储芯片的连接方式。

例4.1 设CPU有16根地址线、8根数据线，并用 $\overline{MREQ}$ 作为访存控制信号（低电平有效），用 $\overline{WR}$ 作为读/写控制信号（高电平为读，低电平为写）。现有下列存储芯片：1 K×4位RAM、4 K×8位RAM、8 K×8位RAM、2 K×8位ROM、4 K×8位ROM、8 K×8位ROM及74138译码器和各种门电路，如图4.36所示。画出CPU与存储器的连接图，要求如下：

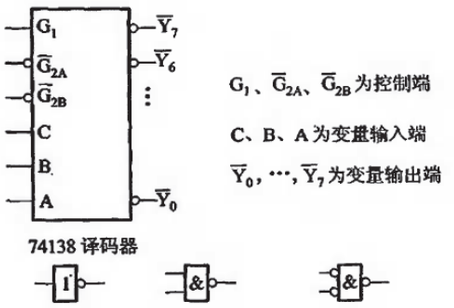


图4.36 译码器和门电路

① 主存地址空间分配：

6000H～67FFH为系统程序区。

6800H～6BFFH为用户程序区。

② 合理选用上述存储芯片，说明各选几片。

③ 详细画出存储芯片的片选逻辑图。

解：第一步，先将十六进制地址范围写成二进制地址码，并确定其总容量。

A_{15} … A_{12}	A_{11} … A_8	A_7 … A_4	A_3 … A_0	
0 1 1 0	0 0 0 0	0 0 0 0	0 0 0 0	系统程序区
…				
0 1 1 0	0 1 1 1	1 1 1 1	1 1 1 1	2 K×8位
0 1 1 0	1 0 0 0	0 0 0 0	0 0 0 0	用户程序区
…				
0 1 1 0	1 0 1 1	1 1 1 1	1 1 1 1	1 K×8位

第二步，根据地址范围的容量以及该范围在计算机系统中的作用，选择存储芯片。

根据6000H～67FFH为系统程序区的范围，应选择1片2 K×8位的ROM，若选择4 K×8位或8 K×8位的ROM，都超出了2 K×8位的系统程序区范围。

根据6800H～6BFFH为用户程序区的范围，选2片1 K×4位的RAM芯片正好满足1 K×8位的用户程序区要求。

第三步，分配CPU的地址线。

将CPU的低11位地址 A_{10}～A_0 与2 K×8位的ROM地址线相连；将CPU的低10位地址 A_9～A_0 与2片1 K×4位的RAM地址线相连。剩下的高位地址与访存控制信号 $\overline{MREQ}$ 共同产生存储芯片的片选信号。

第四步，片选信号的形成。

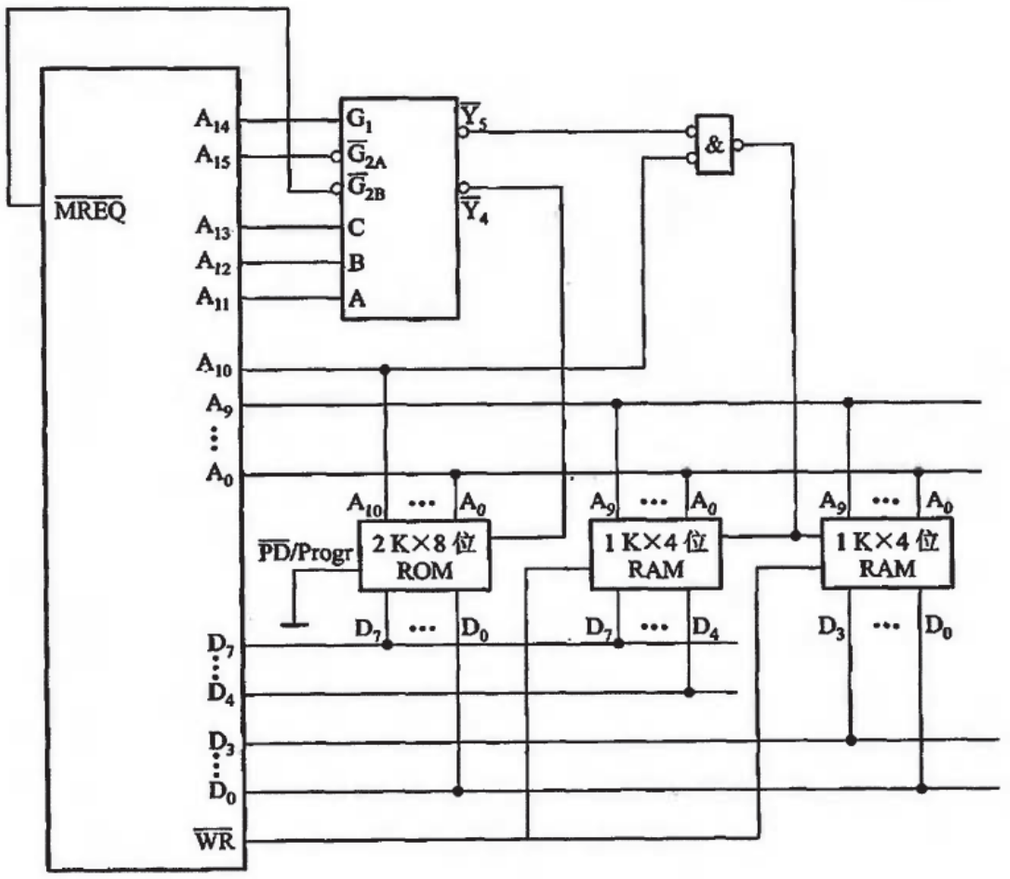


图 4.37　例 4.1 CPU 与存储芯片的连接图

例 4.2　CPU 及其他芯片假设同例 4.1，画出 CPU 与存储器的连接图。要求主存的地址空间满足下述条件：最小 8 K 地址为系统程序区，与其相邻的 16 K 地址为用户程序区，最大 4 K 地址空间为系统程序工作区。详细画出存储芯片的片选逻辑并指出存储芯片的种类及片数。

解:第一步,根据题目的地址范围写出相应的二进制地址码。

A_{15} … A_{12}	A_{11} … A_8	A_7 … A_4	A_3 … A_0	
0 0 0 0	0 0 0 0	0 0 0 0	0 0 0 0	最小 8 K×8 位系统程序区
…				
0 0 0 1	1 1 1 1	1 1 1 1	1 1 1 1	
0 0 1 0	0 0 0 0	0 0 0 0	0 0 0 0	相邻 16 K×8 位用户程序区
…				
0 0 1 1	1 1 1 1	1 1 1 1	1 1 1 1	
0 1 0 0	0 0 0 0	0 0 0 0	0 0 0 0	
…				
0 1 0 1	1 1 1 1	1 1 1 1	1 1 1 1	

A_{15} … A_{12}	A_{11} … A_8	A_7 … A_4	A_3 … A_0	
1 1 1 1	0 0 0 0	0 0 0 0	0 0 0 0	最大 4 K×8 位系统程序工作
…				
1 1 1 1	1 1 1 1	1 1 1 1	1 1 1 1	

第二步,根据地址范围的容量及其在计算机系统中的作用,确定最小 8 KB 系统程序区选择 1 片 8 K×8 位 ROM;与其相邻的 16 KB 用户程序区选择 2 片 8 K×8 位 RAM;最大 4 KB 系统程序工作区选择 1 片 4 K×8 位 RAM。

第三步,分配 CPU 地址线。

将 CPU 的低 13 位地址线 A_{12} ~ A_0 与 1 片 8 K×8 位 ROM 和 2 片 8 K×8 位 RAM 的地址线相连;将 CPU 的低 12 位地址线 A_{11} ~ A_0 与 1 片 4 K×8 位 RAM 的地址线相连。

第四步,形成片选信号

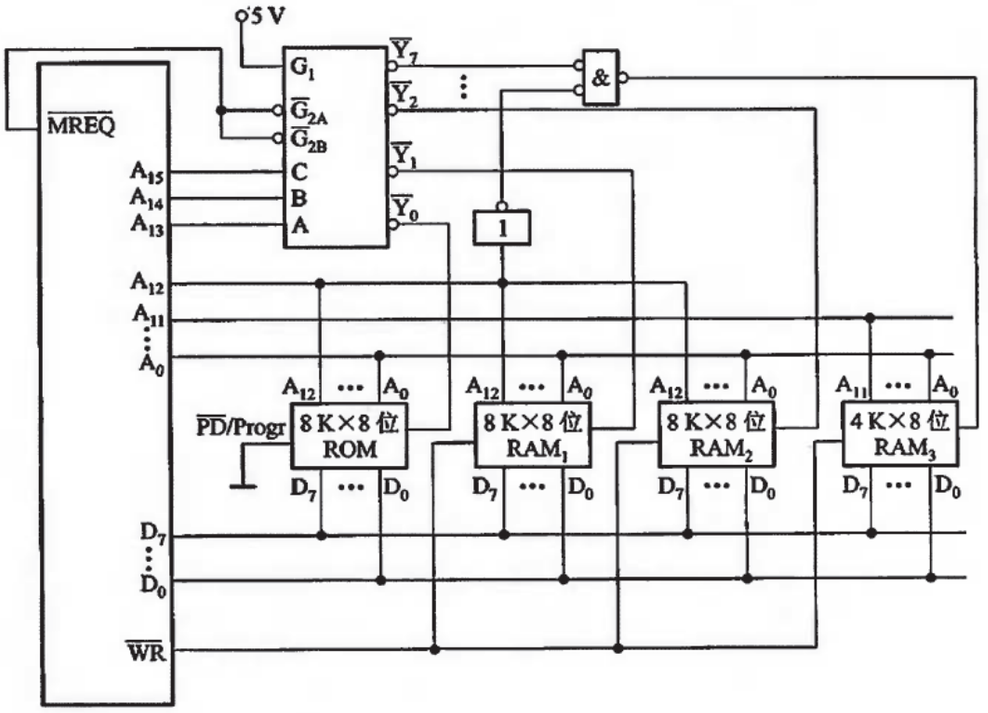


图 4.38　例 4.2 CPU 与存储芯片的连接图

6.Cache 的工作原理（page 111）

（1）贮存和缓存的编址　（2）命中与未命中

（3）Cache 的命中率

H=Nc/(Nc+Nm)　h 为命中率 Nc 为命中次数　Nm 为总命中数

（4）Cache---主存系统的效率

效率 e 与 命中率 有关

$$e=\frac{\text{访问 Cache 的时间}}{\text{平均访问时间}}\times 100\%$$

设 Cache 命中率 为 h，访问 Cache 的时间为 t_c，
访问 主存 的时间为 t_m

则 $$e=\frac{t_c}{h\times t_c+(1-h)\times t_m}\times 100\%$$

例 4.7　假设 CPU 执行某段程序时，共访问 Cache 命中 20 00 次，访问主存 50 次。已知 Cache 的存取周期为 50 ns，主存的存取周期为 200 ns。求 Cache－主存系统的命中率、效率和平均访问时间。

解：(i) Cache 的命中率为

$$2000/(2000+50)=0.97$$

(2) 由题可知，访问主存的时间是访问 Cache 时间的 4 倍(200/50 = 4)。

设访问 Cache 的时间为 t，访问主存的时间为 $4t$，Cache－主存系统的访问效率为 e，则

$$e=\frac{\text{访问 Cache 的时间}}{\text{平均访问时间}}\times 100\%$$

$$=\frac{t}{0.97\times t+(1-0.97)\times 4t}\times 100\%=91.7\%$$

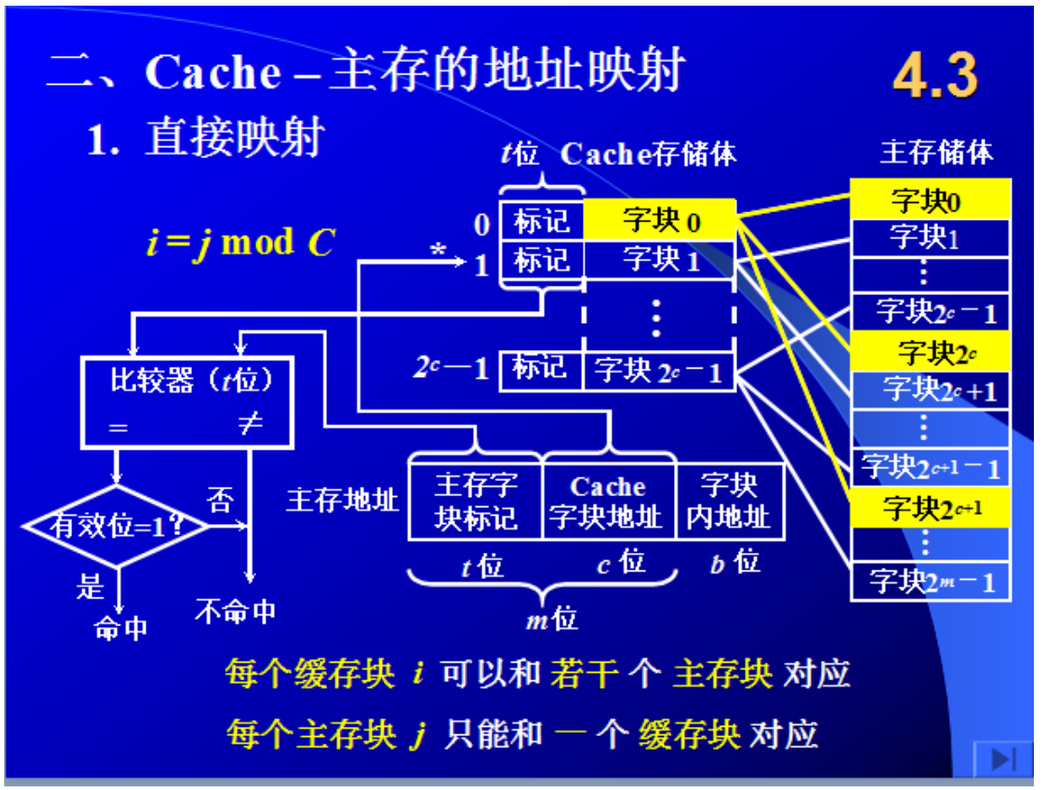
(3) 平均访问时间为

$$50\text{ ns}\times 0.97+200\text{ ns}\times(1-0.97)=54.5\text{ ns}$$

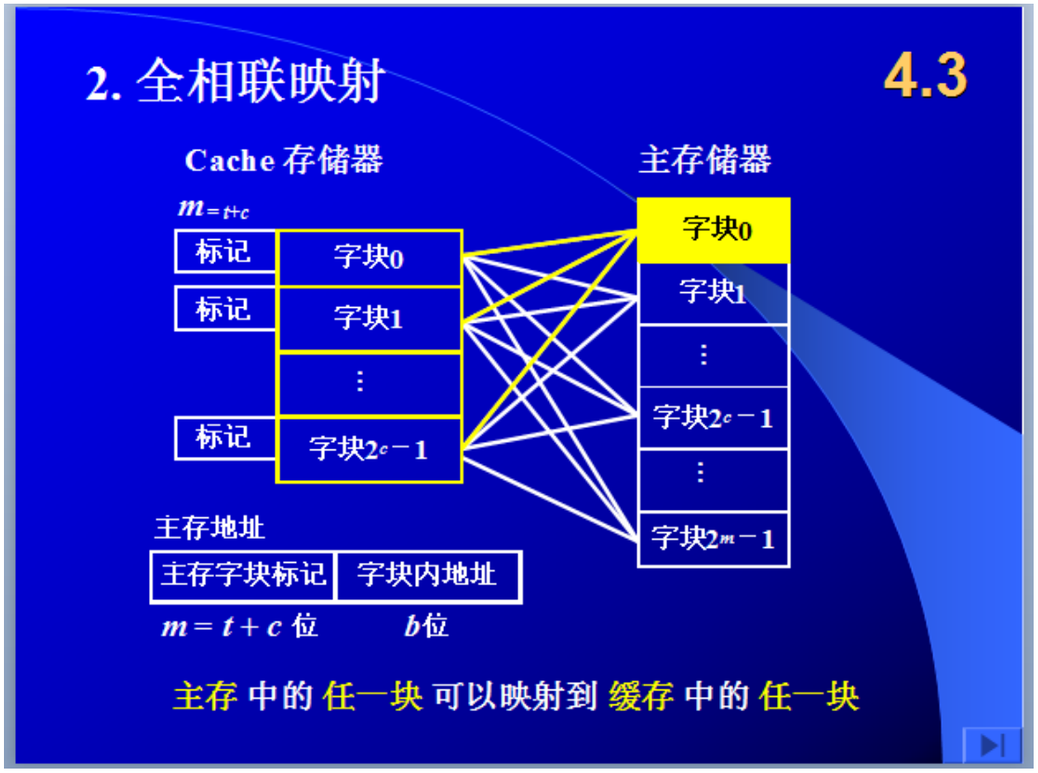
一般而言，Cache 容量越大，其 CPU 的命中率就越高。当然容量也没必要太大，太大会增加成本，而且当 Cache 容量达到一定值时，命中率已不因容量的增大而有明显的提高。因此，Cache

7.Cache-主存地址映射

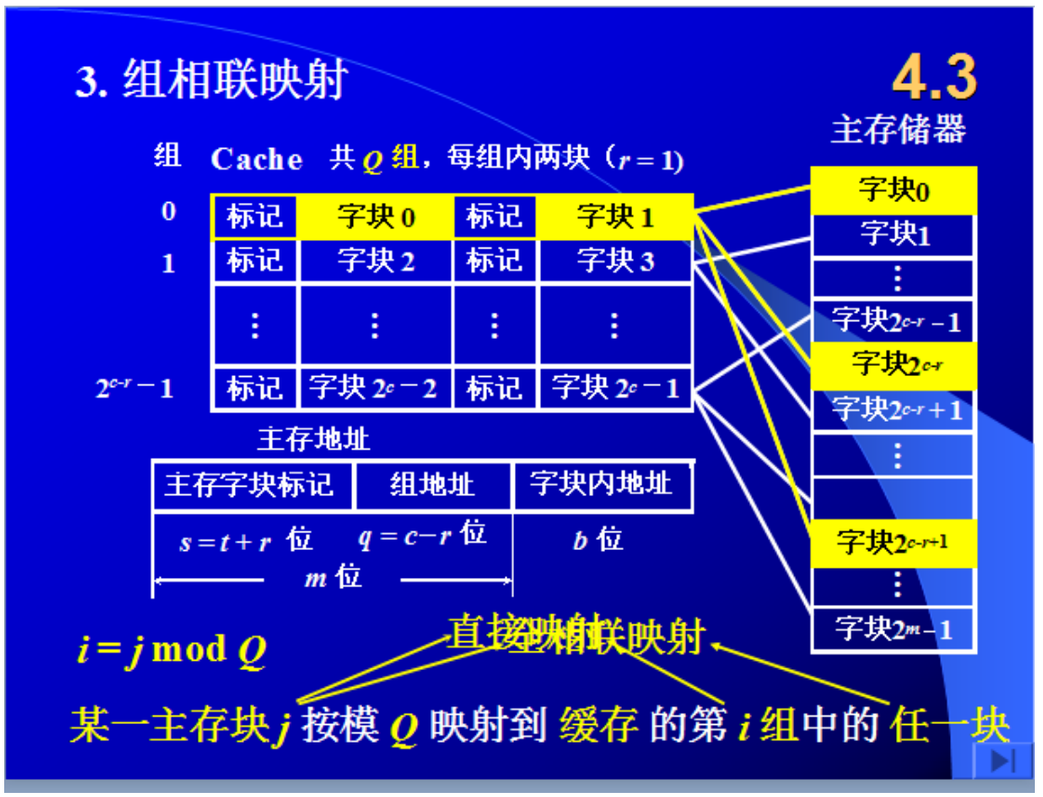
(1)直接映射


二、Cache－主存的地址映射
4.3
1. 直接映射
i=j mod C
t位 Cache存储体
主存储体
0 标记 字块 0
1 标记 字块 1
2c－1 标记 字块 2c－1
*
比较器（t位）
= ≠
有效位=1?
否
是
命中
不命中
主存地址
主存字块标记
Cache字块地址
字块内地址
t 位
c 位
b 位
m位
字块0
字块1
字块2c－1
字块2c
字块2c+1
字块2c+1－1
字块2c+1
字块2m－1
每个缓存块 i 可以和 若干 个 主存块 对应
每个主存块 j 只能和 一 个 缓存块 对应

（2） 全相联映射


2. 全相联映射
4.3
Cache 存储器
主存储器
m = t+c
标记
字块0
标记
字块1
标记
字块2^c − 1
字块0
字块1
字块2^c − 1
字块2^m − 1
主存地址
主存字块标记
字块内地址
m = t + c 位
b位
主存 中的 任一块 可以映射到 缓存 中的 任一块

（3） 组相联映射



例 4.8 假设主存容量为 512 KB，Cache 容量为 4 KB，每个字块为 16 个字，每个字 32 位。

(1) Cache 地址有多少位？可容纳多少块？

(2) 主存地址有多少位？可容纳多少块？

(3) 在直接映射方式下，主存的第几块映射到 Cache 中的第 5 块(设起始字块为第 1 块)？

(4) 画出直接映射方式下主存地址字段中各段的位数。

解：(1) 根据 Cache 容量为 4 KB(2^{12} = 4 K)，Cache 地址为 12 位。由于每字 32 位，则 Cache 共有 4 KB/4 B = 1 K 字。因每个字块 16 个字，故 Cache 中有 1 K/16 = 64 块。

(2) 根据主存容量为 512 KB(2^{19} = 512 K)，主存地址为 19 位。由于每字 32 位，则主存共有 512 KB/4 B = 128 K 字。因每个字块 16 个字，故主存中共 128 K/16 = 8 192 块。

(3) 在直接映射方式下，由于 Cache 共有 64 块，主存共有 8 192 块，因此主存的 5，64 + 5，2 × 64 + 5，…，2^{13} − 64 + 5 块能映射到 Cache 的第 5 块中。

(4) 在直接映射方式下，主存地址字段的各段位数分配如图 4.57 所示。其中字块内地址为 6 位(4 位表示 16 个字，2 位表示每字 32 位)，缓存共 64 块，故缓存字块地址为 6 位，主存字块标记为主存地址长度与 Cache 地址长度之差，即 19 − 12 = 7 位。

主存字块标记	缓存字块地址	字块内地址
7 位	6 位	6 位

例 4.9 假设主存容量为 512 K×16 位，Cache 容量为 4 096×16 位，块长为 4 个 16 位的字，访存地址为字地址。

(1) 在直接映射方式下，设计主存的地址格式。

(2) 在全相联映射方式下，设计主存的地址格式。

(3) 在二路组相联映射方式下，设计主存的地址格式。

(4) 若主存容量为 512 K×32 位，块长不变，在四路组相联映射方式下，设计主存的地址格式。

解：(1) 根据 Cache 容量为 $4\,096 = 2^{12}$ 字，得 Cache 字地址为 12 位。根据块长为 4，且访存地址为字地址，得字块内地址为 2 位，即 $b = 2$，且 Cache 共有 $4\,096/4 = 1\,024 = 2^{10}$ 块，即 $c = 10$。根据主存容量为 $512\text{ K} = 2^{19}$ 字，得主存字地址为 19 位。在直接映射方式下，主存字块标记为 $19 - 12 = 7$。主存的地址格式如图 4.58(a)所示。

(2) 在全相联映射方式下，主存字块标记为 $19 - b = 19 - 2 = 17$ 位，其地址格式如图 4.58(b)所示。

(3) 根据二路组相联的条件，一组内有 2 块，得 Cache 共分 $1\,024/2 = 512 = 2^q$ 组，即 $q = 9$，主存字块标记为 $19 - q - b = 19 - 9 - 2 = 8$ 位，其地址格式如图 4.58(c)所示。

(4) 若主存容量改为 512 K×32 位，即双字宽存储器，块长仍为 4 个 16 位的字，访存地址仍为字地址，则主存容量可写为 1 024 K×16 位，得主存地址为 20 位。由四路组相联，得 Cache 共分 $1\,024/4 = 256 = 2^q$ 组，即 $q = 8$。对应该条件下，主存字块标记为 $20 - 8 - 2 = 10$ 位，其地址格式如图 4.58(d)所示。

主存字块标记	Cache 字块地址	字块内地址
7	10	2

(a) 直接映射方式主存地址格式

主存字块标记	字块内地址
17	2

(b) 全相联映射方式主存地址格式

主存字块标记	组地址	字块内地址
8	9	2

(c) 二路组相联映射方式主存地址格式

主存字块标记	组地址	字块内地址
10	8	2

(d) 四路组相联映射方式双字宽主存地址格式

例 4.10 假设 Cache 的工作速度是主存的 5 倍，且 Cache 被访问命中的概率为 95%，则采用 Cache 后，存储器性能提高多少？

解：设 Cache 的存取周期为 t，主存的存取周期为 $5t$，则系统的平均访问时间为

$$t_a = 0.95 \times t + 0.05 \times 5t = 1.5t$$

性能为原来的 $5t/1.5t = 3.33$ 倍，即提高了 2.33 倍。

例 4.11 设某机主存容量为 16 MB，Cache 的容量为 8 KB。每字块有 8 个字，每字 32 位。设计一个四路组相联映射的 Cache 组织。

（1）画出主存地址字段中各段的位数。

（2）设 Cache 初态为空，CPU 依次从主存第 0，1，2，…，99 号单元读出 100 个字（主存一次读出一个字），并重复此次序读 10 次，问命中率是多少？

（3）若 Cache 的速度是主存速度的 5 倍，试问有 Cache 和无 Cache 相比，速度提高多少倍？

（4）系统的效率为多少？

解：（1）根据每个字块有 8 个字，每个字 32 位，得出主存地址字段中字块内地址字段为 5 位，其中 3 位为字地址，2 位为字节地址。

根据 Cache 容量为 8 KB $= 2^{13}$ B，字块大小为 2^5 B，得 Cache 共有 2^8 块，故 $c = 8$。根据四路组相联映射 $2^r = 4$，得 $r = 2$，则 $q = c - r = 8 - 2 = 6$ 位。

根据主存容量为 16 MB $= 2^{24}$ B，得出主存地址字段中主存字块标记为 $24 - 6 - 5 = 13$ 位。

主存地址字段各段格式如图 4.59 所示。

主存字块标记	组地址	字块内地址
13 位	6 位	5 位

图 4.59 例 4.10 主存地址字段

（2）由于每个字块中有 8 个字，而且初态 Cache 为空，因此 CPU 读第 0 号单元时，未命中，必须访问主存，同时将该字所在的主存块调入 Cache 第 0 组中的任一块内，接着 CPU 读 1 ~ 7 号单元时均命中。同理，CPU 读第 8，16，…，96 号单元时均未命中。可见 CPU 在连续读 100 个字中共有 13 次未命中，而后 9 次循环读 100 个字全部命中，命中率为

$$\frac{100 \times 10 - 13}{100 \times 10} \approx 0.987$$

（3）根据题意，设主存存取周期为 $5t$，Cache 的存取周期为 t，没有 Cache 的访问时间为 $5t \times 1\,000$，有 Cache 的访问时间为 $t\,(1\,000 - 13) + 5t \times 13$，则有 Cache 和没有 Cache 相比，速度提高的倍数为

$$\frac{5t \times 1\,000}{t(1\,000 - 13) + 5t \times 13} - 1 \approx 3.75$$

（4）根据（2）求得的命中率 0.987，主存的存取周期为 $5t$，Cache 的存取周期为 t，得系统的效率为

$$\frac{t}{0.987 \times t + (1 - 0.987) \times 5t} \times 100\% = 95\%$$

8.磁盘存储器的主要技术指标

（1）记录密度

2. 磁表面存储器的主要技术指标

(1) 记录密度

记录密度通常是指单位长度内所存储的二进制信息量。磁盘存储器用道密度和位密度表示;磁带存储器则用位密度表示。磁盘沿半径方向单位长度的磁道数为道密度,单位是 tpi (Track Per Inch ,道每英寸)或 tpm(道每毫米)。为了避免干扰,磁道与磁道之间需保持一定距离,相邻两条磁道中心线之间的距离称为道距,因此道密度 D_t 等于道距 P 的倒数,即

$$D_t = \frac{1}{P}$$

单位长度磁道能记录二进制信息的位数,称为位密度或线密度,单位是 bpi(Bits Per Inch,位每英寸)或 bpm(位每毫米)。磁带存储器主要用位密度来衡量,常用的磁带有 800 bpi、1 600 bpi、6 250 bpi 等。对于磁盘,位密度 D_b 可按下式计算:

$$D_b = \frac{f_t}{\pi d_{min}}$$

其中,f_t 为每道总位数,d_{min}为同心圆中的最小直径。

在磁盘各磁道上所记录的信息量是相同的,而位密度不同,一般泛指磁盘位密度时,是指最内圈磁道上的位密度(最大位密度)。

(2)存储容量

(2) 存储容量

存储容量是指外存所能存储的二进制信息总数量,一般以位或字节为单位。以磁盘存储器为例,存储容量可按下式计算:

$$C = n \times k \times s$$

其中,C 为存储总容量,n 为存放信息的盘面数,k 为每个盘面的磁道数,s 为每条磁道上记录的二进制代码数。

磁盘有格式化容量和非格式化容量两个指标。非格式化容量是磁表面可以利用的磁化单元总数。格式化容量是指按某种特定的记录格式所能存储信息的总量,即用户可以使用的容量,它一般为非格式化容量的 60% ~70% 。

(3) 平均寻址时间

由存取方式分类可知,磁盘采取直接存取方式,寻址时间分为两个部分,其一是磁头寻找目标磁道的找道时间 t_s,其二是找到磁道后,磁头等待欲读/写的磁道区段旋转到磁头下方所需要的等待时间 t_w。由于从最外圈磁道找到最里圈磁道和寻找相邻磁道所需时间是不等的,而且磁头等待不同区段所花的时间也不等,因此,取其平均值,称为平均寻址时间 T_a,它是平均找道时间 t_{sa}和平均等待时间 t_{wa}之和:

$$T_a = t_{sa} + t_{wa} = \frac{t_{smax} + t_{smin}}{2} + \frac{t_{wmax} + t_{wmin}}{2}$$

平均寻址时间是磁盘存储器的一个重要指标。硬磁盘的平均寻址时间比软磁盘的平均寻址时间短,所以硬磁盘存储器比软磁盘存储器速度快。

磁带存储器采取顺序存取方式,磁头不动,磁带移动,不需要寻找磁道,但要考虑磁头寻找记录区段的等待时间,所以磁带寻址时间是指磁带空转到磁头应访问的记录区段所在位置的时间。

(4) 数据传输率

数据传输率 D_r 是指单位时间内磁表面存储器向主机传送数据的位数或字节数，它与记录密度 D_b 和记录介质的运动速度 V 有关：

$$D_r = D_b \times V$$

此外，辅存和主机的接口逻辑应有足够快的传送速度，用来完成接收/发送信息，以便主机与辅存之间正确无误地传送。

(5) 误码率

误码率是衡量磁表面存储器出错概率的参数，它等于从辅存读出时，出错信息位数和读出信息的总位数之比。为了减少出错率，磁表面存储器通常采用循环冗余码来发现并纠正错误。

9.硬盘的记录格式及其例题（了解）

定长记录格式与不定长记录格式

(1) 定长记录格式

一个具有 n 个盘片的磁盘组，可将其 n 个面上同一半径的磁道看成一个圆柱面，这些磁道存储的信息称为柱面信息。在移动磁头组合盘中，磁头定位机构一次定位的磁道集合正好是一个柱面。信息的交换通常在圆柱面上进行，柱面个数正好等于磁道数，故柱面号就是磁道号，而磁头号则是盘面号。

盘面又分若干扇区，每条磁道被分割成若干个扇段，数据在盘片上的布局如图 4.70 所示。扇段是磁盘寻址的最小单位。在定长记录格式中，当台号决定后，磁盘寻址定位首先确定柱面，再选定磁头，最后找到扇段。因此寻址用的磁盘地址应由台号、磁道号、盘面号、扇段号等字段组成，也可将扇段号用扇区号代替。

例 4.12 假设磁盘存储器共有 6 个盘片，最外两侧盘面不能记录，每面有 204 条磁道，每条磁道有 12 个扇段，每个扇段有 512 B，磁盘机以 7 200 rpm 速度旋转，平均定位时间为 8 ms。

(1) 计算该磁盘存储器的存储容量。

(2) 计算该磁盘存储器的平均寻址时间。

解:(1) 6 个盘片共有 10 个记录面,磁盘存储器的总容量为 512 B × 12 × 204 × 10 = 12 533 760 B

(2) 磁盘存储器的平均寻址时间包括平均寻道时间和平均等待时间。其中,平均寻道时间即平均定位时间为 8 ms,平均等待时间与磁盘转速有关。根据磁盘转速为 7 200 rpm,得磁盘每转一周的平均时间为

$$[60\ \text{s}/(7\ 200\ \text{rpm})] \times 0.5 \approx 4.165\ \text{ms}$$

故平均寻址时间为

$$8\ \text{ms} + 4.165\ \text{ms} = 12.165\ \text{ms}$$

例 4.13 一个磁盘组共有 11 片,每片有 203 道,数据传输率为 983 040 Bps,磁盘组转速为 3 600 rpm。假设每个记录块有 1 024 B,且系统可挂 16 台这样的磁盘机,计算该磁盘存储器的总容量并设计磁盘地址格式。

解:(1) 由于数据传输速率 = 每一条磁道的容量 × 磁盘转速,且磁盘转速为 3 600 rpm = 60 rps,故每一磁道的容量为 983 040 Bps/60 rps = 16 384 B。

(2) 根据每个记录块(即扇段)有 1 024 B,故每个磁道有 16 384 B/1 024 B = 16 个扇段。

(3) 磁盘地址格式如图 4.72 所示。其中;台号 4 位,表示有 16 台磁盘机;磁道号 8 位,能反映 203 道;盘面号 5 位,对应 11 个盘片共有 20 个记录面;扇段号 4 位,对应 16 个扇段。

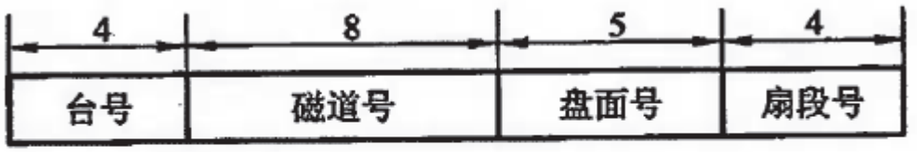


图 4.72 例 4.13 磁盘地址格式

例 4.14 对于一个由 6 个盘面组成的磁盘存储器,若某个文件长度超过一个磁道的容量,应将它记录在同一个存储面上,还是记录在同一个柱面上?

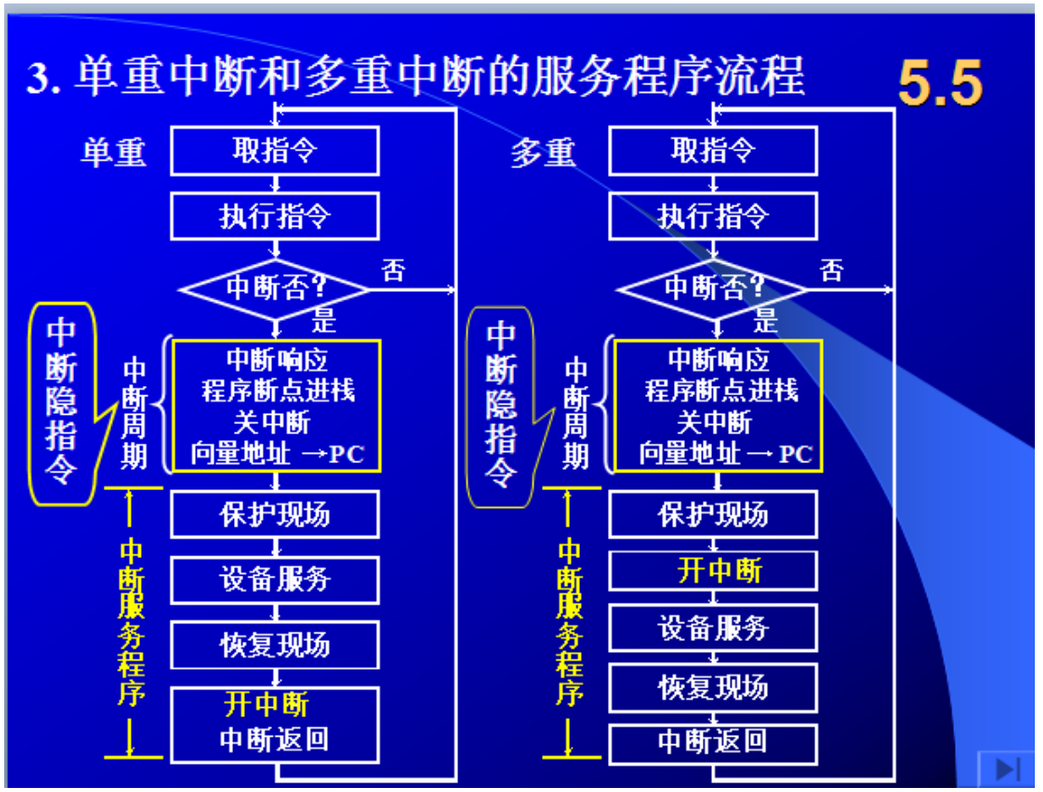
解:如果文件长度超过一个磁道的容量,应将它记录在同一柱面上,因为不需要重新找道,寻址时间减少,数据读/写速度快。

第五章（含第八章的中断）

1.I/O 设备与主机信息传送的控制方式有几种

（1）程序查询方式；（2）程序中断方式；（3）DMA 方式。

2.掌握多重中断与单重中断的流程



3.中断隐指令功能

（1）保存程序断点　（2）寻找中断服务程序的入口地址　（3）关中断

4.中断屏蔽技术

中断屏蔽技术主要用于多重中断

（1）屏蔽触发器与屏蔽字 （2）屏蔽技术可改变优先级

（3）屏蔽技术的其他作用：给其他程序控制带来更来更大的灵活性

例 8.2 设某机有 4 个中断源 1、2、3、4，其硬件排队优先次序按 1→2→3→4 降序排列，各中断源的服务程序中所对应的屏蔽字如表 8.9 所示。

表 8.9 例 8.2 各中断源对应的屏蔽字

中断源	屏蔽字			
	1	2	3	4
1	1	1	0	1
2	0	1	0	0
3	1	1	1	1
4	0	1	0	1

(1) 给出上述 4 个中断源的中断处理次序。

(2) 若 4 个中断源同时有中断请求，画出 CPU 执行程序的轨迹。

解：(1) 根据表 8.9，4 个中断源的处理次序是按 3→1→4→2 降序排列。

(2) 当 4 个中断源同时有中断请求时，由于硬件排队的优先次序是 1→2→3→4，故 CPU 先响应 1 的请求，执行 1 的服务程序。由于在该服务程序中设置了屏蔽字 1101，故开中断指令后转去执行 3 的服务程序，且 3 的服务程序执行结束后又回到 1 的服务程序。1 的服务程序结束后，CPU 还有 2、4 两个中断源请求未响应。由于 2 的响应优先级高于 4，故 CPU 先响应 2 的请求，执行 2 的服务程序。在 2 的服务程序中由于设置了屏蔽字 0100，意味着 1、3、4 可中断 2 的服务程序。而 1、3 的请求已处理结束，因此在开中断指令之后转去执行 4 的服务程序，4 的服务程序执行结束后又回到 2 的服务程序的断点处，继续执行 2 的服务程序，直至该程序执行结束。图 8.37 示意了 CPU 执行程序的轨迹。

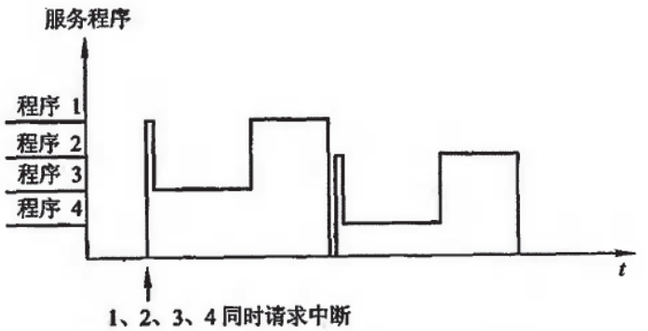


图 8.37 例 8.2 CPU 执行程序的轨迹

5.DMA 方式的特点

首先 DMA 与主存交换数据有三种办法

停止 CPU 访问内存；周期挪用（或周期窃取）；DMA 与 CPU 交替访问。

与程序中断方式相比，DMA 方式的特点：

1 从数据传送上看：程序中断方式靠程序传送，DMA 方式靠硬件传送；

2 从 CPU 响应时间上看，程序中断方式是在一条指令执行结束时响应，而 DMA 方式可在指令周期内任一存取周期结束时响应。

3 程序中断方式有处理异常事件的能力，DMA 则无此能力。

4.程序中断需中断现行程序，故需保护现场，DMA 方式不中断现行程序，无需保护现场。

5.DMA 优先级比程序中断的优先级高。

习题 5.13 和 5.26

13. 说明中断向量地址和入口地址的区别和联系。

解：**中断向量地址和入口地址的区别**：

向量地址是硬件电路（向量编码器）产生的中断源的内存中断向量表表项地址编号，中断入口地址是中断服务程序首址。

中断向量地址和入口地址的联系：

中断向量地址可理解为中断服务程序入口地址指示器（入口地址的地址），通过它访存可获得中断服务程序入口地址。(两种方法：在向量地址所指单元内放一条JUM指令；主存中设向量地址表。

26.什么是多重中断，实现多重中断的必要条件是什么？

解：**多重中断：**当CPU正在执行某个中断服务程序时，另一个中断源又提出了新的中断请求，而CPU又响应了这个新的请求，暂时停止正在运行的服务程序，转去执行新的中断服务程序。

实现多重中断的必要条：

（1）提前设置“开中断”指令 ；

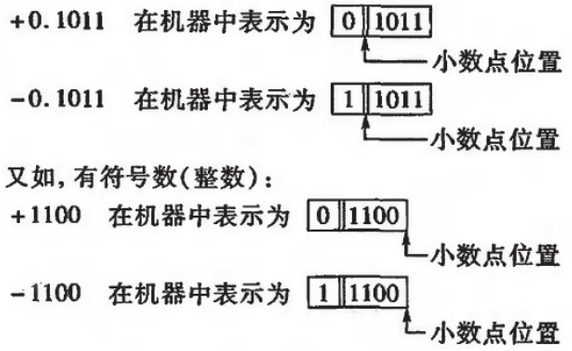
（2）优先级高的中断源有权中断优先级低的中断原。

例 8.2

习题 5.13 和 5.26

第六章

注意 原码、补码的加法一定要会做。这部分主要考虑加减法运算



1.原码、反码、补码和移码的定义和表示范围

原码：

整数原码的定义为

$$[x]_{原}=\begin{cases}0,x & 2^n>x\geqslant 0\\ 2^n-x & 0\geqslant x>-2^n\end{cases}$$

式中，x 为真值，n 为整数的位数。

例如：

当 $x=+1110$ 时，$[x]_{原}=0,1110$

当 $x=-1110$ 时，$[x]_{原}=2^4-(-1110)=1,1110$

↑

用逗号将符号位和数值部分隔开

小数原码的定义为

$$[x]_{原}=\begin{cases}x & 1>x\geqslant 0\\ 1-x & 0\geqslant x>-1\end{cases}$$

式中，x 为真值。

例如：

当 $x=0.1101$ 时，$[x]_{原}=0.1101$

当 $x=-0..1101$ 时，$[x]_{原}=1-(-0.1101)=1.1101$

根据定义，已知真值可求原码，反之已知原码也可求真值。例如：

当 $[x]_{原}=1.0011$ 时，由定义得

$$x=1-[x]_{原}=1-1.0011=-0.0011$$

当 $[x]_{原}=1,1100$ 时，由定义得

$$x=2^n-[x]_{原}=2^4-1,1100=10000-11100=-1100$$

当 $[x]_{原}=0.1101$ 时，$x=0.1101$

当 $x=0$ 时

$$[+0.0000]_{原}=0.0000$$

$$[-0.0000]_{原}=1-(0.0000)=1.0000$$

可见 $[+0]_{原}$ 不等于 $[-0]_{原}$，即原码中的“零”有两种表示形式。

补码：

(2) 补码的定义

整数补码的定义为

$$[x]_{补}=\begin{cases}0,x & 2^n>x\geqslant 0\\ 2^{n+1}+x & 0>x\geqslant -2^n\end{cases}\quad(\mathrm{mod}\ 2^{n+1})$$

式中，x 为真值，n 为整数的位数。

例如：

当 $x=+1010$ 时，

$[x]_{补}=0,1010$

↑

用逗号将符号位和数值部分隔开

当 $x=-1101$ 时，

$[x]_{补}=2^{n+1}+x=100000-1101=1,0011$

↑

用逗号将符号位和数值部分隔开

小数补码的定义为

$$[x]_{补}=\begin{cases}x & 1>x\geqslant 0\\ 2+x & 0>x\geqslant -1\end{cases}\quad(\mathrm{mod}\ 2)$$

式中，x 为真值。

例如：

当 $x=0.1001$ 时，

$[x]_{补}=0.1001$

当 $x=-0.0110$ 时，

$[x]_{补}=2+x=10.0000-0.0110=1.1010$

当 $x=0$ 时，

$[+0.0000]_{补}=0.0000$

$[-0.0000]_{补}=2+(-0.0000)=10.0000-0.0000=0.0000$

显然 $[+0]_{补}=[-0]_{补}=0.0000$，即补码中的“零”只有一种表示形式。

反码：

4. 反码表示法

反码通常用来作为由原码求补码或者由补码求原码的中间过渡。反码的定义如下：

整数反码的定义为

$$[x]_{反}=\begin{cases}0,x & 2^n>x\geqslant 0\\ (2^{n+1}-1)+x & 0\geqslant x>-2^n\end{cases}\quad(\mathrm{mod}(2^{n+1}-1))$$

式中，x 为真值，n 为整数的位数。

例如：

当 $x=+1101$ 时，

$[x]_{反}=0,1101$

↑

用逗号将符号位和数值部分隔开

当 $x=-1101$ 时，

$[x]_{反}=(x^{4+1}-1)+x=11111-1101=1,0010$

↑

用逗号将符号位和数值部分隔开

小数反码的定义为

$$[x]_{反}=\begin{cases}x & 1>x\geqslant 0\\ (2-2^{-n})+x & 0\geqslant x>-1\end{cases}\quad(\mathrm{mod}(2-2^{-n}))$$

式中，x 为真值，n 为小数的位数。

例如：

当 $x = +0.0110$ 时，

$$[x]_{反} = 0.0110$$

当 $x = -0.0110$ 时，

$$[x]_{反} = (2 - 2^{-4}) + x = 1.1111 - 0.0110 = 1.1001$$

当 $x = 0$ 时，

$$[+0.0000]_{反} = 0.0000$$

$$[-0.0000]_{反} = (10.0000 - 0.0001) - 0.0000 = 1.1111$$

可见 $[+0]_{反}$ 不等于 $[-0]_{反}$，即反码中的“零”也有两种表示形式。

移码：

由此可得移码的定义

$$[x]_{移} = 2^n + x \quad (2^n > x \geqslant -2^n)$$

式中，x 为真值，n 为整数的位数。

例如：$x=10100$

$$[x]_{移}=2^5+10100=1,10100$$

↑

用逗号将符号位和数值部分隔开

$$x=-10100$$

$$[x]_{移}=2^5-10100=0,01100$$

↑

用逗号将符号位和数值部分隔开

当 $x=0$ 时

$$[+0]_{移}=2^5+0=1,00000$$

$$[-0]_{移}=2^5-0=1,00000$$

可见$[+0]_{移}$等于$[-0]_{移}$，即移码表示中零也是惟一的。

表 6.2　真值、补码和移码对照表

真值 x	$[x]_{补}$	$[x]_{移}$	$[x]_{移}$对应的十进制整数
-100000	100000	000000	0
-11111	100001	000001	1
-11110	100010	000010	2
⋮	⋮	⋮	⋮
-00001	111111	011111	31
±00000	000000	100000	32
+00001	000001	100001	33
+00010	000010	100010	34
⋮	⋮	⋮	⋮
+11110	011110	111110	62
+11111	011111	111111	63

2.原码、补码的加法

1. 补码加减运算的基本公式

补码加法的基本公式如下：

整数　$[A]_{补}+[B]_{补}=[A+B]_{补}(\bmod 2^{n+1})$

小数　$[A]_{补}+[B]_{补}=[A+B]_{补}(\bmod 2)$

则$[A-B]_{补}=[A+(-B)]_{补}$

由补码加法基本公式可得

整数　$[A-B]_{补}=[A]_{补}+[-B]_{补}(\mathrm{mod}\ 2^{n+1})$

小数　$[A-B]_{补}=[A]_{补}+[-B]_{补}(\mathrm{mod}\ 2)$

因此，若机器数采用补码，当求 $A-B$ 时，只需先求 $[-B]_{补}$（称 $[-B]_{补}$ 为“求补”后的减数），就可按补码加法规则进行运算。而 $[-B]_{补}$ 由 $[B]_{补}$ 连同符号位在内，每位取反，末位加 1 而得。

例 6.8　已知 $A=0.1011, B=-0.0101$，求 $[A+B]_{补}$。

解：因为　$A=0.1011, B=-0.0101$

所以　$[A]_{补}=0.1011, [B]_{补}=1.1011$

则　$[A]_{补}+[B]_{补}=0.1011$

$$\begin{array}{r} +1.1011 \\ \hline \boxed{1}\ 0.0110=[A+B]_{补} \end{array}$$

丢掉

按模 2 的意义，最左边的 1 丢掉，故 $[A+B]_{补}=0.0110$，结果正确。

例 6.9　已知 $A=-1001, B=-0101$，求 $[A+B]_{补}$。

解：因为　$A=-1001, B=-0101$

所以　$[A]_{补}=1,0111, [B]_{补}=1,1011$

则　$[A]_{补}+[B]_{补}=1,0111$

$$\begin{array}{r} +1,1011 \\ \hline \boxed{1}\ 1,0010=[A+B]_{补} \end{array}$$

丢掉

按模 2^{4+1} 的意义，最左边的 1 丢掉。

例 6.10　设机器数字长为 8 位（含 1 位符号位），若 $A=+15, B=+24$，求 $[A-B]_{补}$ 并还原成真值。

解：因为　$A=+15=+0001111, B=+24=+0011000$

所以　$[A]_{补}=0,0001111, [B]_{补}=0,0011000, [-B]_{补}=1,1101000$

则　$[A-B]_{补}=[A]_{补}+[-B]_{补}=0,0001111$

$$\begin{array}{r} +1,1101000 \\ \hline 1,1110111 \end{array}$$

所以　$[A-B]_{补}=1,1110111$

故　$A-B=-0001001=-9$

可见，不论操作数是正还是负，在做补码加减法时，只需将符号位和数值部分一起参加运算，并且将符号位产生的进位自然丢掉即可。

例 6.12 已知 $A=-\frac{11}{16}, B=-\frac{7}{16}$，求 $[A+B]_{补}$。

解：由 $A=-\frac{11}{16}=-0.1011, B=-\frac{7}{16}=-0.0111$

得 $[A]_{补}=1.0101, [B]_{补}=1.1001$

所以

$$\begin{array}{rl} [A+B]_{补}=[A]_{补}= & 1.0101 \\ +[B]_{补}= & 1.1001 \\ \hline & \boxed{1}\ 0.1110 \end{array}$$

丢掉

两操作数符号均为 1，结果的符号为 0，故为溢出。

例 6.13 已知 $A=-0.1000, B=-0.1000$，求 $[A+B]_{补}$。

解：由 $A=-0.1000, B=-0.1000$

得 $[A]_{补}=1.1000, [B]_{补}=1.1000$

所以

$$\begin{array}{rl} [A+B]_{补}=[A]_{补}= & 1.1000 \\ +[B]_{补}= & 1.1000 \\ \hline & \boxed{1}\ 1.0000 \end{array}$$

丢掉

结果的符号同原操作数符号，故未溢出。

由 $[A+B]_{补}=1.0000$，得 $A+B=-1$，由此可见，用补码表示定点小数时，它能表示 -1 的值。

计算机中采用 1 位符号位判断时，为了节省时间，通常用符号位产生的进位与最高有效位产生的进位异或操作后，按其结果进行判断。若异或结果为 1，即为溢出；异或结果为 0，则无溢出。例 6.12 中符号位有进位，最高有效位无进位，即 $1\oplus 0=1$，故溢出。例 6.13 中符号位有进位，最高有效位也有进位，即 $1\oplus 1=0$，故无溢出。

3.掌握加减法，了解乘除

4.浮点数的表示范围

2. 浮点数的表示范围

以通式 $N=S\times r^{j}$ 为例，设浮点数阶码的数值位取 m 位，尾数的数值位取 n 位，当浮点数为非规格化数时，它在数轴上的表示范围如图 6.2 所示。

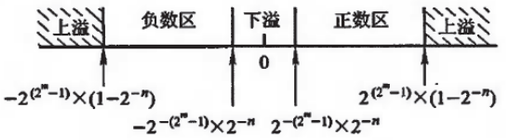


图 6.2　浮点数在数轴上的表示范围

由图中可见，其最大正数为 $2^{(2^m-1)}\times(1-2^{-n})$；最小正数为 $2^{-(2^m-1)}\times2^{-n}$；最大负数为 $-2^{-(2^m-1)}\times2^{-n}$；最小负数为 $-2^{(2^m-1)}\times(1-2^{-n})$。当浮点数阶码大于最大阶码时，称为上溢，此时机器停止运算，进行中断溢出处理；当浮点数阶码小于最小阶码时，称为下溢，此时溢出的数绝对值很小，通常将尾数各位强置为零，按机器零处理，此时机器可以继续运行。

3. 浮点数的规格化

为了提高浮点数的精度，其尾数必须为规格化数。如果不是规格化数，就要通过修改阶码并同时左右移尾数的办法，使其变成规格化数。将非规格化数转换成规格化数的过程称为规格化。对于基数不同的浮点数，因其规格化数的形式不同，规格化过程也不同。

当基数为 2 时，尾数最高位为 1 的数为规格化数。规格化时，尾数左移一位，阶码减 1（这种规格化称为向左规格化，简称左规）；尾数右移一位，阶码加 1（这种规格化称为向右规格化，简称右规）。图 6.2 所示的浮点数规格化后，其最大正数为 $2^{(2^m-1)}\times(1-2^{-n})$，最小正数为 $2^{-(2^m-1)}\times2^{-1}$；最大负数为 $-2^{-(2^m-1)}\times2^{-1}$，最小负数为 $-2^{(2^m-1)}\times(1-2^{-n})$。

当基数为 4 时，尾数的最高两位不全为零的数为规格化数。规格化时，尾数左移两位，阶码减 1；尾数右移两位，阶码加 1。

当基数为 8 时，尾数的最高三位不全为零的数为规格化数。规格化时，尾数左移三位，阶码减 1；尾数右移三位，阶码加 1。

同理类推，不难得到基数为 16 或 2^n 时的规格化过程。

5.浮点和真值的相互转换

例 6.3　设浮点数字长 16 位，其中阶码 5 位（含 1 位阶符），尾数 11 位（含 1 位数符），将十进制数 $+\frac{13}{128}$ 写成二进制定点数和浮点数，并分别写出它在定点机和浮点机中的机器数形式。

解：令 $x=+\frac{13}{128}$

其二进制形式　　$x=0.0001101000$

定点数表示　　$x=0.0001101000$

浮点数规格化表示　　$x=0.1101000000\times2^{-11}$

定点机中　$[x]_{原}=[x]_{补}=[x]_{反}=0.0001101000$

浮点机中

$[x]_{原}$：

1	0011	0	1101000000

或写成1,0011;0.1101000000

$[x]_{补}$：

1	1101	0	1101000000

或写成1,1101;0.1101000000

$[x]_{反}$：

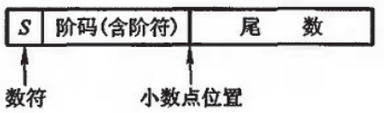
1	1100	0	1101000000

或写成1,1100;0.1101000000

6.IEEE754 标准

6.2.5 IEEE 754 标准

现代计算机中，浮点数一般采用 IEEE 制定的国际标准，这种标准形式如下：

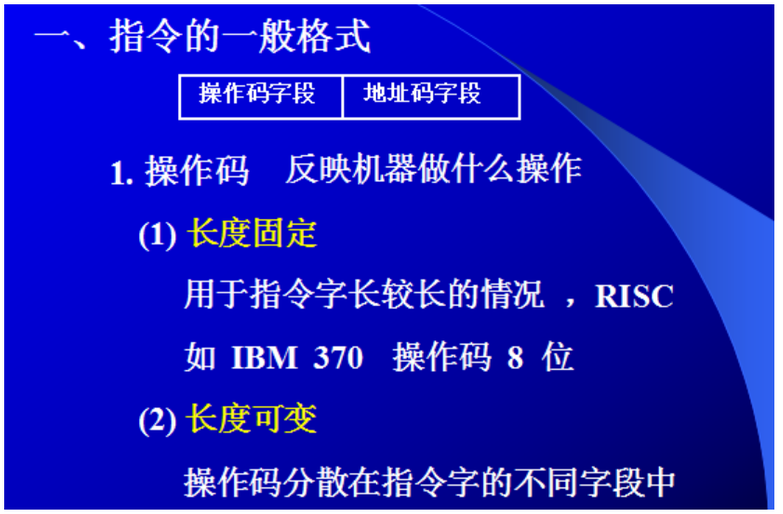


按 IEEE 标准，常用的浮点数有三种：

	符号位 S	阶码	尾数	总位数
短实数	1	8	23	32
长实数	1	11	52	64
临时实数	1	15	64	80

第七章

1.指令的一般格式



例 7.1

例 7.1 假设指令字长为 16 位，操作数的地址码为 6 位，指令有零地址、一地址、二地址三种格式。

(1) 设操作码固定，若零地址指令有 P 种，一地址指令有 Q 种，则二地址指令最多有几种？

(2) 采用扩展操作码技术，若二地址指令有 X 种，零地址指令有 Y 种，则一地址指令最多有几种？

解：(1) 根据操作数地址码为 6 位，则二地址指令中操作码的位数为 $16-6-6=4$。这 4 位操作码可有 $2^4=16$ 种操作。由于操作码固定，则除去了零地址指令 P 种，一地址指令 Q 种，剩下二地址指令最多有 $16-P-Q$ 种。

(2) 采用扩展操作码技术，操作码位数可变，则二地址、一地址和零地址的操作码长度分别为 4 位、10 位和 16 位。可见二地址指令操作码每减少一种，就可多构成 2^6 种一地址指令操作码；一地址指令操作码每减少一种，就可多构成 2^6 种零地址指令操作码。

因二地址指令有 X 种，则一地址指令最多有 $(2^4-X)\times 2^6$ 种。设一地址指令有 M 种，则零地址指令最多有 $[(2^4-X)\times 2^6-M]\times 2^6$ 种。

根据题中给出零地址指令有 Y 种，即

$$Y=[(2^4-X)\times 2^6-M]\times 2^6$$

则一地址指令

$$M=(2^4-X)\times 2^6-Y\times 2^{-6}$$

在设计操作码不固定的指令系统时，应尽量考虑安排指令使用频度(即指令在程序中出现的概率)高的指令占用短的操作码，对使用频度低的指令可占用较长的操作码，这样可以缩短经常使用的指令的译码时间。当然，考虑操作码长度时也应考虑地址码的要求。

2.指令寻址方式分类

顺序寻址和跳跃寻址

3.数据寻址方式分类

立即寻址；直接寻址；隐含寻址；间接寻址；寄存器寻址；寄存器间接寻址；基址寻址；变址寻址；相对寻址；堆栈寻址。

4.RISC 和 CISC 的比较

（1）充分利用 VLSI 芯片的面积

（2）提高计算机运算速度

（3）便于设计，可降低成本，提高可靠性

（4）有效支持高级语言程序

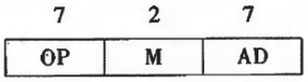
例 7.5 和 7.6

例 7.5 设某机配有基址寄存器和变址寄存器，采用一地址格式的指令系统，允许直接和间接寻址，且指令字长、机器字长和存储字长均为 16 位。

（1）若采用单字长指令，共能完成 105 种操作，则指令可直接寻址的范围是多少？一次间接寻址的寻址范围是多少？画出其指令格式并说明各字段的含义。

（2）若存储字长不变，可采用什么方法直接访问容量为 16 MB 的主存？

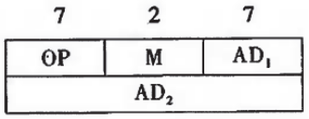
解：（1）在单字长指令中，根据能完成 105 种操作，取操作码 7 位。因允许直接和间接寻址，且有基址寄存器和变址寄存器，故取 2 位寻址特征位，其指令格式如下：



其中，OP 为操作码，可完成 105 种操作；M 为寻址特征，可反映四种寻址方式；AD 为形式地址。

这种指令格式可直接寻址 $2^7=128$，一次间接寻址的寻址范围是 $2^{16}=65\ 536$。

（2）容量为 16 MB 的存储器，正好与存储字长为 16 位的 8 M 存储器容量相等，即 16 MB = 8 M × 16 位。欲使指令直接访问 16 MB 的主存，可采用双字长指令，其操作码和寻址特征位均不变，其格式如下：



其中，形式地址为 $AD_1 // AD_2$，共 7 + 16 = 23 位。$2^{23}=8$ M，即可直接访问主存的任一位置。

例 7.6 某模型机共有 64 种操作，操作码位数固定，且具有以下特点。

(1) 采用一地址或二地址格式。

(2) 有寄存器寻址、直接寻址和相对寻址(位移量为 -128 ~ +127)三种寻址方式。

(3) 有 16 个通用寄存器，算术运算和逻辑运算的操作数均在寄存器中，结果也在寄存器中。

(4) 取数/存数指令在通用寄存器和存储器之间传送数据。

(5) 存储器容量为 1 MB，按字节编址。

要求设计算逻指令、取数/存数指令和相对转移指令的格式，并简述理由。

解：(1) 算逻指令格式为寄存器 - 寄存器型，取单字长 16 位。

6	2	4	4
OP	M	R_i	R_j

其中，OP 为操作码，6 位，可实现 64 种操作；M 为寻址模式，2 位，可反映寄存器寻址、直接寻址、相对寻址；R_i 和 R_j 各取 4 位，指出源操作数和目的操作数的寄存器(共 16 个)编号。

(2) 取数/存数指令格式为寄存器 - 存储器型，取双字长 32 位，格式如下：

6	2	4	4
OP	M	R_i	A_1
A_2			

其中，OP 为操作码，6 位不变；M 为寻址模式，2 位不变；R_i 为 4 位，源操作数地址(存数指令)或目的操作数地址(取数指令)；A_1 和 A_2 共 20 位，为存储器地址，可直接访问按字节编址的 1 MB 存储器。

(3) 相对转移指令为一地址格式，取单字长 16 位，格式如下：

6	2	8
OP	M	A

其中，OP 为操作码，6 位不变；M 为寻址模式，2 位不变；A 为位移量 8 位，对应位移量为 -128 ~ +127。

第八章

1.CPU 的功能

取指令、分析指令、执行指令

2.指令周期的基本概念

CPU 每取出并执行一条指令所需的全部时间成为指令周期，也即 CPU 完成一条指令的时间

3.影响流水线性能的因素

结构相关、数据相关、控制相关

4.流水线的性能指标

吞吐率、加速比、效率

第九章

1.微操作命令中四个周期的内容

四个周期为（取指周期、间址周期、执行周期、中断周期），具体内容见书 375-378

2.多级时序系统

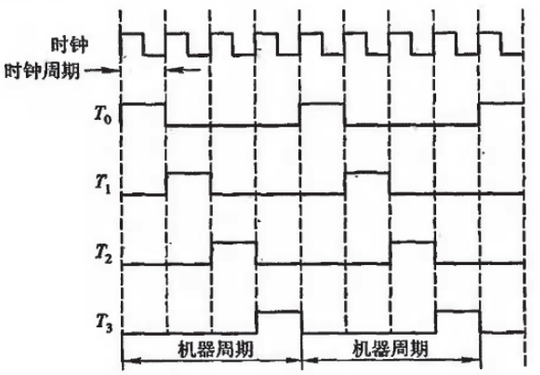
1. 机器周期

机器周期可看做是所有指令执行过程中的一个基准时间，机器周期取决于指令的功能及器件的速度。确定机器周期时，通常要分析机器指令的执行步骤及每一步骤所需的时间。例如，取数、存数指令能反映存储器的速度及其与 CPU 的配合情况；加法指令能反映 ALU 的速度；条件转移指令因为要根据上一条指令的执行结果，经测试后才能决定是否转移，所需的时间较长。总之，通过对机器指令执行步骤的分析，会找到一个基准时间，在这个基准时间内，所有指令的操作都能结束。若以这个基准时间定为机器周期，显然不是最合理的。因为只有以完成复杂指令功能所需的时间（最长时间）作为基准，才能保证所有指令在此时间内完成全部操作，这对简单指令来说，显然是一种浪费。

进一步分析发现，机器内的各种操作大致可归属为对 CPU 内部的操作和对主存的操作两大类，由于 CPU 内部的操作速度较快，CPU 访存的操作时间较长，因此通常以访问一次存储器的时间定为基准时间较为合理，这个基准时间就是机器周期。又由于不论执行什么指令，都需要访问存储器取出指令，因此在存储字长等于指令字长的前提下，取指周期也可看做机器周期。

2. 时钟周期(节拍、状态)

在一个机器周期里可完成若干个微操作,每个微操作都需要一定的时间,可用时钟信号来控制产生每一个微操作命令(如图 9.3 中的 C_i)。时钟就好比计算机的心脏,只要接通电源,计算机内就会产生时钟信号。时钟信号可由机器主振电路(如晶体振荡器)发出的脉冲信号经整形(或倍频、分频)后产生,时钟信号的频率即为 CPU 主频。用时钟信号控制节拍发生器,就可产生节拍。每个节拍的宽度正好对应一个时钟周期。在每个节拍内机器可完成一个或几个需同时执行的操作,它是控制计算机操作的最小时间单位。图 9.8 反映了机器周期、时钟周期和节拍的关系,图中一个机器周期内有 4 个节拍 T_0、T_1、T_2、T_3。



↓96.6K/S ↑432K/S

3. 多级时序系统

图 9.9 反映了指令周期、机器周期、节拍(状态)和时钟周期的关系。可见,一个指令周期包含若干个机器周期,一个机器周期又包含若干个时钟周期(节拍),每个指令周期内的机器周期数可以不等,每个机器周期内的节拍数也可以不等。其中,图 9.9(a)为定长的机器周期,每个机器周期包含 4 个节拍(4 个 T);图 9.9(b)为不定长的机器周期,每个机器周期包含的节拍数可以为 4 个,也可以为 3 个,后者适合于操作比较简单的指令,它可跳过某些时钟周期(如 T_3),从而缩短指令周期。

机器周期、节拍(状态)组成了多级时序系统。

一般来说,CPU 的主频越快,机器的运行速度也越快。在机器周期所含时钟周期数相同的前提下,两机平均指令执行速度之比等于两机主频之比。例如,CPU 的主频为 8 MHz,其平均指令执行速度为 0.8 MIPS。若想得到平均指令执行速度为 0.4 MIPS 的机器,则只需要用主频为 (8 MHz × 0.4 MIPS)/0.8 MIPS = 4 MHz 的 CPU 即可。

实际上机器的速度不仅与主频有关,还与机器周期中所含的时钟周期数以及指令周期中所

机器周期与节拍(状态)组成了多级时序系统

3.指令周期、机器周期和时钟周期(上面)

例 9.3 设某计算机的 CPU 主频为 8 MHz，每个机器周期平均含 2 个时钟周期，每条指令的指令周期平均有 2.5 个机器周期，试问该机的平均指令执行速度为多少 MIPS？若 CPU 主频不变，但每个机器周期平均含 4 个时钟周期，每条指令的指令周期平均有 5 个机器周期，则该机的平均指令执行速度又是多少 MIPS？由此可得出什么结论？

解：由于主频为 8 MHz，所以时钟周期为 1/8 = 0.125 μs，机器周期为 0.125 × 2 = 0.25 μs，指令周期为 0.25 × 2.5 = 0.625 μs。

① 平均指令执行速度为 1/0.625 = 1.6 MIPS。

② 若 CPU 主频不变，机器周期含 4 个时钟周期，每条指令平均含 5 个机器周期，则指令周期为 0.125 × 4 × 5 = 2.5 μs，故平均指令执行速度为 1/2.5 = 0.4 MIPS。

③ 可见机器的运行速度并不完全取决于主频。

此外，机器的运行速度还和其他很多因素有关，如主存的运行速度、机器是否配有 Cache、总线的数据传输率、硬盘的运行速度以及机器是否采用流水技术等。机器速度还可以用 MIPS（执行百万条指令数每秒）和 CPI（执行一条指令所需的时钟周期数）来衡量。

例 9.3

第十章

1.控制单元设计的设计方法

组合逻辑设计和微程序设计

2.组合逻辑控制

（1）列出微操作命令的操作时间表 ；

（2）写出微操作命令的最简逻辑表达式 ；

（3）画出微操作命令的逻辑图

3.微指令的编码方式

（1）直接编码（直接控制）方式

（2）字段直接编码方式

（3）字段间接编码方式

（4）混合编码

（5）其他（如：设置常数字段）

例 10.4,10.5 和 10.6

例 10.4　某机的微指令格式中，共有 8 个控制字段，每个字段可分别激活 5、8、3、16、1、7、25、4 种控制信号。分别采用直接编码和字段直接编码方式设计微指令的操作控制字段，并说明两种方式的操作控制字段各取几位。

解：(1) 采用直接编码方式，微指令的操作控制字段的总位数等于控制信号数，即

$$5+8+3+16+1+7+25+4=69$$

(2) 采用字段直接编码方式，需要的控制位少。根据题目给出的 10 个控制字段及各段可激活的控制信号数，再加上每个控制字段至少要留一个码字表示不激活任何一条控制线，即微指令的 8 个控制字段分别需给出 6、9、4、17、2、8、26、5 种状态，对应 3、4、2、5、1、3、5、3 位，故微指令的操作控制字段的总位数为

$$3+4+2+5+1+3+5+3=26$$

例 10.5 某微程序控制器中，采用水平型直接控制（编码）方式的微指令格式，后续微指令地址由微指令的下地址字段给出。已知机器共有 28 个微命令、6 个互斥的可判定的外部条件，控制存储器的容量为 512 ×40 位。试设计其微指令格式，并说明理由。

解：水平型微指令由操作控制字段、判别测试字段和下地址字段三部分构成。因为微指令采用直接控制（编码）方式，所以其操作控制字段的位数等于微命令数，为 28 位。又由于后续微指令地址由下地址字段给出，故其下地址字段的位数可根据控制存储器的容量（512 ×40 位）定为 9 位。当微程序出现分支时，后续微指令地址的形成取决于状态条件，6 个互斥的可判定外部条件，可以编码成 3 位状态位。非分支时的后续微指令地址由微指令的下地址字段直接给出。微指令的格式如图 10.12 所示。

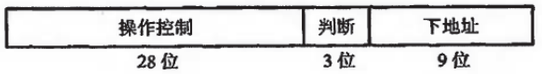


图 10.12 例 10.5 微指令格式

例 10.6 某机共有 52 个微操作控制信号，构成 5 个相斥类的微命令组，各组分别包含 5、8、2、15、22 个微命令。已知可判定的外部条件有两个，微指令字长 28 位。

（1）按水平型微指令格式设计微指令，要求微指令的下地址字段直接给出后续微指令地址。

（2）指出控制存储器的容量。

解：（1）根据 5 个相斥类的微命令组，各组分别包含 5、8、2、15、22 个微命令，考虑到每组必须增加一种不发命令的情况，条件测试字段应包含一种不转移的情况，则 5 个控制字段分别需给出 6、9、3、16、23 种状态，对应 3、4、2、4、5 位（共 18 位），条件测试字段取 2 位。根据微指令字长为 28 位，则下地址字段取 28 − 18 − 2 = 8 位，其微指令格式如图 10.13 所示。

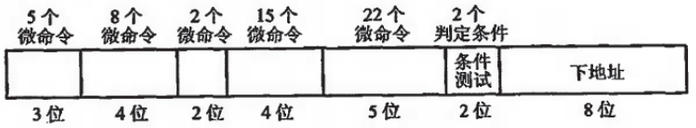


图 10.13 例 10.6 微指令格式

（2）根据下地址字段为 8 位，微指令字长为 28 位，得控制存储器的容量为 256 × 28 位